Transformer-Based CI Build Failure Prediction on TravisTorrent: A Fair Comparison with Parrot, BuildFast, and DL-CIBuild

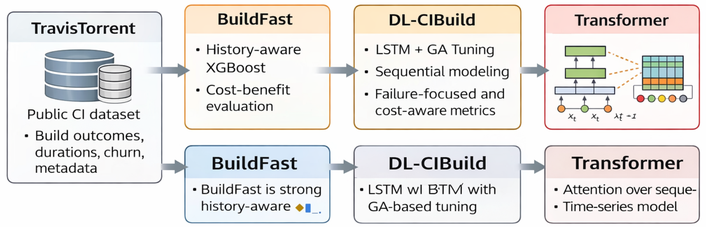
Wilson Neira

Northeastern University

Background

Continuous Integration (CI) systems automatically build and test code after changes, but builds can take a long time and failed builds slow developers down. My project studies whether a Transformer-based time-series model can improve CI build failure prediction compared with strong baselines.

- TravisTorrent gives a public CI dataset with build outcomes, durations, churn, and metadata.
- BuildFast is a strong history-aware XGBoost baseline with cost-benefit evaluation.
- DL-CIBuild treats build prediction as a sequence problem using LSTM + GA tuning.



Problem / Research Question

My main question is: can a Transformer-based sequential model improve CI build failure prediction on TravisTorrent compared with BuildFast-style, DL-CIBuild-style, and Parrot baselines?

- Focus on failure-class recall and F1, not just plain accuracy.
- Keep the comparison fair by using the same common10 project subset.
- Check practical value using benefit, cost, and gain in build hours.

Methods

I built a fair common10 benchmark so the models are compared on the same 10 projects instead of accidentally using different project subsets.

- Preprocess TravisTorrent-style build history and create history-aware features.
- Build tabular inputs for BuildFast and sequence windows for LSTM / Transformer.
- Evaluate all models with the same failure-focused and cost-aware metrics.

Methods Overview

Fair common10 benchmark: same 10 projects, same evaluation metrics, different model families

1. Data
- TravisTorrent-style CI build history
- Public metadata: outcomes, durations, churn, commit info
- Keep only the so every 10 projects so every model uses the same project subset

2. Preprocessing
- Sort builds chronologically within each project
- Build tabular features for BuildFast-style models
- Build sequence windows for LSTM and Transformer
- Use fair train / validation / test splits and the same target label

3. Models
- Parrot: copy previous build outcome
- BuildFast: history-aware tabular model
- DL-CIBuild / SharedSeq LSTM: sequence model
- Transformer: attention over past build windows

4. Evaluation
- Failure-focused metrics: precision, recall, F1-fail
- Overall metrics: accuracy, ROC AUC, PR AUC
- Practical metrics: benefit, cost, gain in build hours
- Compare all rows on the same common10 benchmark

Why this is fair

All models are evaluated on the same common10 project subset. The main difference is the model family: Parrot uses a simple prior, BuildFast uses history-aware tabular features, and LSTM / Transformer use sequence windows.

- Parrot copies the previous build outcome.
- BuildFast uses history-aware tabular modeling.
- DL-CIBuild uses LSTM with GA-based tuning.
- My Transformer uses sequence windows and attention.

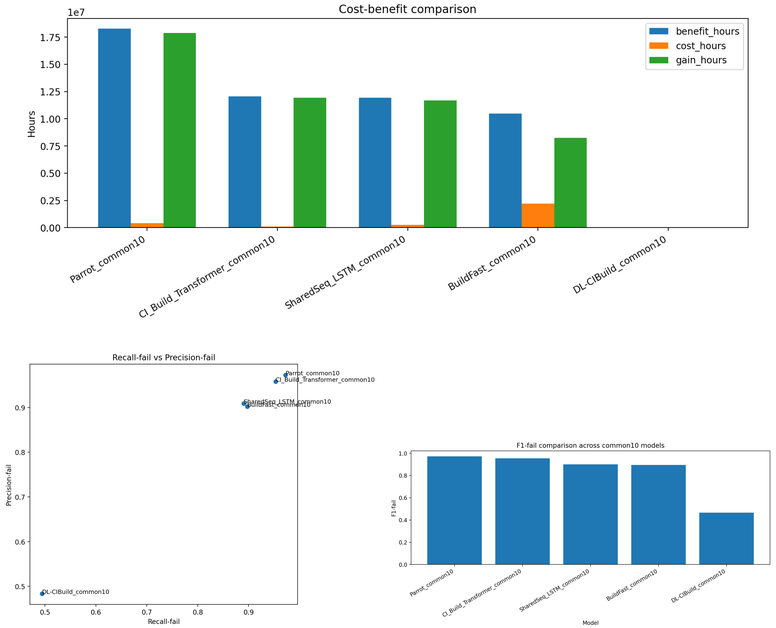
Future work:

- Improve the Transformer on flip cases, where Parrot is less useful.
- Run more ablations on sequence inputs, depth, heads, and threshold tuning.

Results

The main result so far is that Parrot is still the strongest fair common10 row, but my Transformer is currently the best learned sequence model on this benchmark.

- Parrot_common10 is the strongest model on the fair common10 benchmark.
- My Transformer is the best learned sequence model and outperforms SharedSeq LSTM, BuildFast, and DL-CIBuild on this benchmark.
- Even though it does not beat Parrot, the Transformer still looks useful for harder CI prediction cases.



References

[1] Beller, Gousios, Zaidman. *TravisTorrent*. MSR 2017.
Chen et al. *BuildFast*. ASE 2020.
Saidani, Ouni, Mkaouer. *DL-CIBuild*. ASE 2022.
RavenBuild paper for Parrot strength and flip-case motivation.